UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES

SCHOOL OF ENGINEERING SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

CPEN 421- MOBILE AND WEB SOFTWARE ARCHITECTURE

FIRST SEMESTER 2025/2026 ACADEMIC YEAR

LAB REPORT ON PHASE 2

GROUP 18

Contributors:

NAME: NANA KWASI WADIE KWAKYE

STUDENT ID: 11358243

GITHUB LINK: https://github.com/Nanakwasi07/emergencyresponsephase2.git

Table of Contents

1. Executive Summary
2. System Architecture
   2.1 Microservices Overview
   2.2 Communication Patterns
   2.3 RabbitMQ Event Flow
3. Infrastructure & Configuration
   3.1 Environment Configuration
   3.2 Database Strategy
   3.3 Project Structure
4. Shared Common Modules
   4.1 RabbitMQ Messaging Layer
   4.2 Event Constants
   4.3 Utility Helpers
   4.4 Authentication Middleware
5. Auth Service (Port 3001)
6. Incident Service (Port 3002)
7. Dispatch Service (Port 3003)
8. Analytics Service (Port 3004)
9. Data Models Overview
10. Bugs Fixed
11. API Reference Summary
12. Startup & Developer Guide

1. Executive Summary

Phase 2 delivers a fully functional backend for the National Emergency Response Coordination Platform, a distributed system designed to manage emergency incidents, dispatch responders, and provide real-time operational analytics for Ghana's emergency services.

Key Metrics

Metric	Value	Detail
Microservices	4	Auth · Incident · Dispatch · Analytics
REST API Endpoints	34	Across all 4 services
RabbitMQ Events	8	Async inter-service event types
Message Queues	5	Durable topic queues
Bugs Fixed	6	Route, model, config, and DNS issues
Real-time Protocol	Socket.IO	Vehicle location broadcasts

Key Deliverables

- Event-driven architecture using RabbitMQ topic exchange (emergency_platform)
- Real-time vehicle location tracking via Socket.IO
- Self-sufficient Analytics Service powered by consumed events — no upstream HTTP calls
- Station registry for police stations, fire stations, and hospitals with bed capacity
- Swagger UI interactive documentation on every service at /api-docs
- Single shared .env configuration file for all microservices
- MongoDB Atlas with per-service database isolation using the dbName option

2. System Architecture

2.1 Microservices Overview

Service	Port	Database	Responsibility
Auth Service	3001	auth_service	User registration, login, JWT issuance & refresh, role management
Incident Service	3002	incident_service	Emergency incident lifecycle, automatic responder matching using Haversine distance
Dispatch Service	3003	dispatch_service	Vehicle fleet management, GPS tracking, station registry, Socket.IO real-time broadcasts
Analytics Service	3004	analytics_service	Operational dashboards, KPI metrics, incident mapping to Ghana regions — all from local DB populated by RabbitMQ events

2.2 Communication Patterns

The system uses two communication patterns:

- Synchronous HTTP (REST) — Used by the Incident Service to query the Dispatch Service for available vehicles when auto-dispatching. Fast, request/response.
- Asynchronous RabbitMQ events — All cross-service state propagation (incident creation, dispatch confirmation, vehicle status, analytics). Decoupled, durable, and resilient to service downtime.

2.3 RabbitMQ Event Flow

Exchange: emergency_platform (type: topic, durable)

```
Incident Service ──incident.created──────────────────────▶ analytics.incidents
──▶ Analytics
                 ──incident.dispatched──────────────────▶
analytics.incidents
                 ──incident.dispatched──────────────────▶ dispatch.incidents
──▶ Dispatch
                 ──incident.status.updated──────────────▶
analytics.incidents
                 ──incident.resolved────────────────────▶
analytics.incidents

Dispatch Service ──vehicle.location.updated──────────────▶ analytics.vehicles
──▶ Analytics
                 ──vehicle.status.changed───────────────▶ analytics.vehicles
                 ──vehicle.status.changed───────────────▶ incident.vehicles
──▶ Incident

Auth Service     ──user.registered──────────────────────▶ analytics.users
──────▶ Analytics
```

Queue Binding Table

Queue	Binding Key(s)	Consumer	Purpose
analytics.incidents	incident.*	Analytics	All incident lifecycle events
analytics.vehicles	vehicle.*	Analytics	Vehicle location & status events

Queue	Binding Key(s)	Consumer	Purpose
analytics.users	user.*	Analytics	User registration events
dispatch.incidents	incident.dispatched	Dispatch	Auto-mark vehicle as dispatched
incident.vehicles	vehicle.status.changed	Incident	Vehicle availability cache

3. Infrastructure & Configuration

3.1 Environment Configuration

A single root-level .env file is shared across all services. Each service loads it with:

```
require('dotenv').config({ path: require('path').join(__dirname,
'../../../.env') });
```

Key variables:

```
MONGODB_URI=mongodb://user:pass@shard-00-00.clauhrg.mongodb.net:27017,
            shard-00-01.clauhrg.mongodb.net:27017,
            shard-00-02.clauhrg.mongodb.net:27017/
            ?authSource=admin&replicaSet=atlas-xxx-shard-0&tls=true
AUTH_SERVICE_PORT=3001
JWT_SECRET=<strong-secret>          JWT_EXPIRE=24h
JWT_REFRESH_SECRET=<secret>         JWT_REFRESH_EXPIRE=7d
INCIDENT_SERVICE_PORT=3002
DISPATCH_SERVICE_PORT=3003
ANALYTICS_SERVICE_PORT=3004
RABBITMQ_URL=amqps://user:pass@gull.rmq.cloudamqp.com/vhost
RABBITMQ_EXCHANGE=emergency_platform
NODE_ENV=development
```

Note — MongoDB URI Format

Node.js v25 on Windows uses c-ares DNS which cannot resolve mongodb+srv:// SRV records. The URI was converted to a standard mongodb:// URI with all three Atlas shard hostnames explicitly listed, plus replicaSet and tls=true parameters discovered from DNS TXT records.

3.2 Database Strategy

Each microservice connects to its own MongoDB database on the shared Atlas cluster:

Service	Database Name	Key Collections
Auth	auth_service	users
Incident	incident_service	incidents
Dispatch	dispatch_service	vehicles, locationhistories, stations
Analytics	analytics_service	analyticsevents, incidentsummaries, vehiclesummaries

The dbName option in mongoose.connect() assigns each service to its own database:

```
await mongoose.connect(process.env.MONGODB_URI, { dbName: 'auth_service'
});
```

3.3 Project Structure

```
course-project/
├── .env                        ← Single shared environment file
├── package.json                ← Root scripts (dev, start, install:all)
├── common/
│   ├── messaging/
│   │   ├── rabbitmq.js         ← Connection manager + pub/sub
│   │   └── events.js           ← Event constants & queue definitions
│   ├── middleware/
│   │   ├── auth.js             ← JWT verification middleware
│   │   └── errorHandler.js     ← Express error handler
│   └── utils/
│       └── helpers.js          ← Haversine formula, responder matching
└── services/
```

```
    ├── auth-service/           (port 3001)
    ├── incident-service/       (port 3002)
    ├── dispatch-service/       (port 3003)
    └── analytics-service/      (port 3004)
```

4. Shared Common Modules

4.1 RabbitMQ Messaging Layer

File: common/messaging/rabbitmq.js

A singleton connection manager for RabbitMQ with automatic retry and reconnection logic. All four services import this module.

Key behaviours:

- Retry on startup — up to 10 attempts with 3-second delays before giving up
- Auto-reconnect — listens to connection.on("close") and schedules reconnect
- Durable topic exchange — declared on every connection, survives broker restarts
- Prefetch(10) — limits concurrent unacknowledged messages per consumer
- ACK/NACK — messages are acknowledged on success, nacked (no requeue) on handler error

```
// Publish an event
const publish = (routingKey, payload) => {
  // Serialises payload as JSON Buffer
  // Sets persistent: true (survives broker restart)
  // Returns false if channel is not ready
};

// Subscribe a queue to one or more binding keys
const subscribe = async (queueName, bindingKeys, handler) => {
  // Asserts durable queue
  // Binds to each routing key pattern (supports wildcards: *, #)
  // handler signature: async (routingKey, payload) => void
};
```

4.2 Event Constants

File: common/messaging/events.js

Constant	Routing Key	Published By
INCIDENT_CREATED	incident.created	Incident Service
INCIDENT_DISPATCHED	incident.dispatched	Incident Service
INCIDENT_STATUS_UPDATED	incident.status.updated	Incident Service
INCIDENT_RESOLVED	incident.resolved	Incident Service
VEHICLE_REGISTERED	vehicle.registered	Dispatch Service
VEHICLE_LOCATION_UPDATED	vehicle.location.updated	Dispatch Service
VEHICLE_STATUS_CHANGED	vehicle.status.changed	Dispatch Service
USER_REGISTERED	user.registered	Auth Service

4.3 Utility Helpers

File: common/utils/helpers.js

Haversine Distance Formula

Used by the Incident Service to rank vehicles by proximity before dispatching the nearest available one. Returns distance in kilometres between two GPS coordinates.

```
const calculateDistance = (lat1, lon1, lat2, lon2) => {
  const R = 6371; // Earth's radius in km
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a = Math.sin(dLat/2)**2 +
            Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
Math.sin(dLon/2)**2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
};
```

Responder Type Mapping (Bug Fix Applied)

```
const getResponderType = (incidentType) => {
  const typeMap = {
    'robbery': 'police_car',   // was: 'police' (wrong enum value)
    'assault': 'police_car',   // was: 'police'
    'fire':    'fire_truck',   // was: 'fire'
    'medical': 'ambulance',
    'accident':'ambulance'
  };
  return typeMap[incidentType.toLowerCase()] || null;
};
```

4.4 Authentication Middleware

File: common/middleware/auth.js

Verifies Authorization: Bearer <token> on every protected route. Sets req.user with the decoded JWT payload. Returns 401 on missing or invalid token.

5. Auth Service (Port 3001)

5.1 Responsibilities

- User registration with bcrypt password hashing (salt rounds: 10)
- JWT access token (24 h) + refresh token (7 d) issuance
- Token refresh endpoint to renew expired access tokens without re-login
- Role-based access control: admin, hospital_admin, police_admin, fire_admin
- Publishes user.registered event to RabbitMQ after successful registration

5.2 API Endpoints

Method	Path	Auth	Description
POST	`/auth/register`	Public	Register a new user. Required: name, email, password, role. Returns 201 + user object.
POST	`/auth/login`	Public	Authenticate. Returns accessToken, refreshToken, and user object.
POST	`/auth/refresh-token`	Public	Exchange valid refreshToken for a new accessToken.
GET	`/auth/profile`	JWT	Return authenticated user's profile.
GET	`/auth/users`	Admin JWT	List all registered users (passwords excluded).

5.3 User Model

```
{
  name:         String (2-100 chars, required),
  email:        String (unique, lowercase, required),
  passwordHash: String (auto-hashed by pre-save hook),
  role:         Enum
['admin','hospital_admin','police_admin','fire_admin'],
  isActive:     Boolean (default: true),
  lastLogin:    Date,
  timestamps:   createdAt, updatedAt
}
```

5.4 RabbitMQ Event Published

```
// Routing key: user.registered
{
  userId:    "64abc...",
  name:      "John Doe",
  email:     "john@example.com",
  role:      "admin",
  createdAt: "2026-03-19T10:00:00Z"
}
```

6. Incident Service (Port 3002)

6.1 Responsibilities

- Full lifecycle management: Created → Dispatched → In Progress → Resolved
- Automatic nearest-responder matching using Haversine distance against the Dispatch Service vehicle list
- Publishing lifecycle events to RabbitMQ for downstream services to react
- Subscribing to vehicle.status.changed to maintain vehicle availability awareness

6.2 API Endpoints

Method	Path	Auth	Description
POST	`/incidents`	JWT	Create a new incident and trigger automatic responder dispatch.
GET	`/incidents/open`	JWT	List all unresolved incidents sorted by creation date.
GET	`/incidents/all`	JWT	Paginated list of all incidents. Query: page, limit.
GET	`/incidents/:id`	JWT	Get single incident by ID.
PUT	`/incidents/:id/status`	JWT	Update incident status. Auto-calculates responseTime (minutes) on resolution.
PUT	`/incidents/:id/assign`	JWT	Manually trigger responder matching for an existing incident.

6.3 Auto-Dispatch Logic

When an incident is created, matchAndDispatchResponder() runs asynchronously:

- Maps incident type → vehicle type (e.g. fire → fire_truck)
- Queries Dispatch Service: GET /vehicles?type=fire_truck&status=available
- Computes Haversine distance from incident location to each available vehicle
- Selects the closest vehicle and updates the incident's assignedUnit field
- Publishes INCIDENT_DISPATCHED — Dispatch Service marks vehicle busy, Analytics records the event

6.4 RabbitMQ Events Published

Event	Trigger	Key Payload Fields
incident.created	POST /incidents	incidentId, type, location, adminId, status
incident.dispatched	Auto-dispatch success	incidentId, vehicleId, vehicleType, distanceKm
incident.status.updated	PUT /status	incidentId, status, responseTime, resolvedAt
incident.resolved	Status → Resolved	incidentId, responseTime, resolvedAt

6.5 Incident Model

```
{
  citizenName:  String (required),
  incidentType: Enum ['robbery','assault','fire','medical','accident'],
```

```
  latitude:     Number (required),
  longitude:    Number (required),
  notes:        String,
  adminId:      String (required),
  assignedUnit: { vehicleId, vehicleType, serviceType },
  status:       Enum ['Created','Dispatched','In Progress','Resolved'],
  resolvedAt:   Date,
  responseTime: Number (minutes),
  timestamps:   createdAt, updatedAt
}
```

7. Dispatch Service (Port 3003)

7.1 Responsibilities

- Emergency vehicle fleet registry and real-time GPS tracking
- Socket.IO server for live vehicle location push to connected frontend clients
- Station registry: police stations, fire stations, hospitals with bed capacity
- Consumes incident.dispatched events and auto-marks vehicles as dispatched
- Publishes vehicle location and status change events to RabbitMQ

7.2 Real-Time Architecture

The Dispatch Service wraps Express inside a Node.js http.createServer() and passes the same server instance to Socket.IO. This allows both REST and WebSocket traffic on port 3003.

```
const server = http.createServer(app);
const io = new SocketIOServer(server, { cors: { origin: '*' } });
setIO(io); // injects io into vehicle routes

// On vehicle location update:
io.emit('vehicle:location', { vehicleId, latitude, longitude, status,
timestamp });

// On incident.dispatched RabbitMQ event:
io.emit('vehicle:dispatched', { vehicleId, incidentId, vehicleType,
distanceKm });
```

7.3 Vehicle API Endpoints

Method	Path	Auth	Description
POST	`/vehicles/register`	JWT	Register a new vehicle. Default status: available.
GET	`/vehicles`	JWT	List vehicles. Filter by ?type= and ?status=.
GET	`/vehicles/:id`	JWT	Get vehicle details.
GET	`/vehicles/:id/location`	JWT	Get current GPS coordinates.
POST	`/vehicles/:id/location`	None	Update GPS (called from driver app). Saves to LocationHistory, broadcasts via Socket.IO, publishes event.
PUT	`/vehicles/:id/status`	None	Update vehicle status. Increments totalIncidentsResponded when returning to available.
GET	`/vehicles/:id/location-history`	JWT	Last N GPS records (default 50).

7.4 Station API Endpoints

Method	Path	Auth	Description
POST	`/stations`	JWT	Register a new station (police/fire/hospital).
GET	`/stations`	JWT	List active stations. Filter by ?type= and ?region=.
GET	`/stations/:id`	JWT	Get station details.

Method	Path	Auth	Description
PUT	`/stations/:id`	JWT	Update station information.
PUT	`/stations/:id/capacity`	JWT	Update hospital bed capacity (totalBeds, availableBeds).

7.5 Data Models

```
// Vehicle
{
  vehicleType: Enum ['ambulance','police_car','fire_truck'],
  serviceId: String, driverId: String, driverName: String,
  latitude: Number, longitude: Number,
  status: Enum
['available','dispatched','on_scene','in_transit','unavailable'],
  currentIncidentId: String,
  lastLocationUpdate: Date,
  totalIncidentsResponded: Number
}

// LocationHistory (separate collection)
{ vehicleId: ObjectId (ref Vehicle), latitude, longitude, timestamp }

// Station
{
  name: String, stationType: Enum ['police','fire','hospital'],
  latitude: Number, longitude: Number,
  address: String, region: String, contactPhone: String,
  totalBeds: Number,     // hospitals only
  availableBeds: Number, // hospitals only
  isActive: Boolean
}
```

8. Analytics Service (Port 3004)

8.1 Design Philosophy — Event-Sourced Analytics

The Analytics Service is entirely decoupled from other services at query time. It consumes RabbitMQ events as they happen and maintains its own aggregated view in analytics_service MongoDB. All dashboard and reporting queries hit local collections — no upstream HTTP calls, no auth dependency, no coupling to other services' databases.

> **Key Architectural Benefit**
> If the Incident Service or Dispatch Service goes offline, the Analytics Service continues to serve cached metrics from its last known state. It will catch up automatically when services come back online and resume publishing events.

8.2 Event Consumer

File: services/analytics-service/src/messaging/consumer.js

Subscribes to two queues at startup. For every event received:

- Persists the raw event to AnalyticsEvent collection (full audit log)
- Upserts a summarised document in IncidentSummary or VehicleSummary

Event	Action
incident.created	Insert IncidentSummary (incidentId, type, location, adminId, status)
incident.dispatched	Update status → Dispatched, set assignedUnit
incident.status.updated	Update status, responseTime, resolvedAt
incident.resolved	Update status → Resolved, record responseTime
vehicle.location.updated	Upsert VehicleSummary, increment totalLocationUpdates
vehicle.status.changed	Update vehicleType, status, lastSeen in VehicleSummary

8.3 API Endpoints

Method	Path	Auth	Description
GET	`/analytics/dashboard`	JWT	Comprehensive summary: total/resolved/open incidents, avg response time, type breakdown, vehicle status.
GET	`/analytics/response-times`	JWT	Average response time overall and per incident type.
GET	`/analytics/incidents-by-region`	JWT	Incident counts mapped to Ghana's 16 regions using lat/lon bounding boxes.
GET	`/analytics/incidents-by-type`	JWT	Incident count per type: robbery, assault, fire, medical, accident.
GET	`/analytics/resource-utilization`	JWT	Vehicle status distribution by type (ambulance/police_car/fire_truck).
GET	`/analytics/most-deployed`	JWT	Top 10 most active vehicles ranked by totalLocationUpdates.

Method	Path	Auth	Description
GET	`/analytics/events`	JWT	Raw event log. Filter: ?type=incident.created&limit=50.

8.4 Analytics Models

```
// AnalyticsEvent — raw event audit log
{ eventType: String (indexed), payload: Mixed, processedAt: Date }
// Index: { eventType: 1, createdAt: -1 }

// IncidentSummary — pre-aggregated per incident
{
  incidentId: String (unique), incidentType: String, citizenName: String,
  latitude: Number, longitude: Number, adminId: String, status: String,
  responseTime: Number, createdAt: Date, resolvedAt: Date, assignedUnit:
Mixed
}
// Indexes: incidentType, status, createdAt

// VehicleSummary — pre-aggregated per vehicle
{
  vehicleId: String (unique), vehicleType: String, status: String,
  lastLatitude: Number, lastLongitude: Number,
  lastSeen: Date, totalLocationUpdates: Number
}
```

9. Data Models Overview

Service	Model	Key Fields	Notes
Auth	User	name, email, passwordHash, role, isActive, lastLogin	bcrypt pre-save hook; toJSON strips passwordHash
Incident	Incident	citizenName, incidentType, lat/lon, adminId, assignedUnit, status, responseTime, resolvedAt	responseTime computed from createdAt on resolution
Dispatch	Vehicle	vehicleType, serviceId, driverId, lat/lon, status, currentIncidentId, totalIncidentsResponded	status enum enforced; counter incremented on return-to-available
Dispatch	LocationHistory	vehicleId (ref), latitude, longitude, timestamp	Separate collection; appended on every GPS update
Dispatch	Station	name, stationType, lat/lon, region, contactPhone, totalBeds, availableBeds, isActive	totalBeds/availableBeds relevant for hospitals only
Analytics	AnalyticsEvent	eventType, payload, processedAt	Full audit log of all RabbitMQ events received
Analytics	IncidentSummary	incidentId (unique), type, status, responseTime, lat/lon	Upserted on every incident.* event
Analytics	VehicleSummary	vehicleId (unique), type, status, lastLat/Lon, totalLocationUpdates	Upserted on every vehicle.* event

10. Bugs Fixed

Bug 1 — Route Shadowing (incident-service)

GET /incidents/open was declared after GET /incidents/:id, causing Express to treat "open" as a document ID and return 404 or a CastError. Fix: Moved all static routes (/open, /all) before the /:id parameterized route.

Bug 2 — Use-Before-Define in Status Update

In PUT /:id/status, responseTime was calculated using incident.createdAt before the variable incident was assigned (it was set by the result of findByIdAndUpdate). Fix: Fetch the incident with findById first, compute responseTime, then update.

Bug 3 — Responder Type Enum Mismatch

getResponderType() returned "police" and "fire" but the Vehicle model enum uses "police_car" and "fire_truck". Vehicle queries always returned empty arrays for police and fire incidents. Fix: Updated the type map in helpers.js to use correct enum values.

Bug 4 — Analytics Service Unauthenticated HTTP Calls

Original analytics routes made HTTP GET calls to Incident/Dispatch service endpoints which require JWT authentication, always receiving 401. Fix: Analytics service now queries its own local MongoDB collections, populated via RabbitMQ event consumption. Zero upstream HTTP dependency.

Bug 5 — MongoDB URI Corruption

Original code appended /dbname directly to a URI that already had /?appName=... query params, producing a malformed URI. Fix: Removed the URI manipulation entirely; uses the dbName option in mongoose.connect() instead.

Bug 6 — Node.js v25 DNS SRV Resolution Failure

Node.js v25 uses c-ares DNS resolver which cannot perform SRV lookups on Windows, causing "querySrv ECONNREFUSED" for mongodb+srv:// URIs. Fix: Replaced mongodb+srv:// URI with a standard mongodb:// URI using all three Atlas shard hostnames and the replicaSet parameter discovered from DNS TXT records.

11. API Reference Summary

Swagger UI interactive documentation is available at /api-docs on each service port:

Service	Swagger URL
Auth	http://localhost:3001/api-docs
Incident	http://localhost:3002/api-docs
Dispatch	http://localhost:3003/api-docs
Analytics	http://localhost:3004/api-docs

Full Endpoint Inventory (34 endpoints)

#	Method	Path	Service	Auth
1	POST	`/auth/register`	Auth	Public
2	POST	`/auth/login`	Auth	Public
3	POST	`/auth/refresh-token`	Auth	Public
4	GET	`/auth/profile`	Auth	JWT
5	GET	`/auth/users`	Auth	Admin JWT
6	GET	`/health`	Auth	Public
7	POST	`/incidents`	Incident	JWT
8	GET	`/incidents/open`	Incident	JWT
9	GET	`/incidents/all`	Incident	JWT
10	GET	`/incidents/:id`	Incident	JWT
11	PUT	`/incidents/:id/status`	Incident	JWT
12	PUT	`/incidents/:id/assign`	Incident	JWT
13	GET	`/health`	Incident	Public
14	POST	`/vehicles/register`	Dispatch	JWT
15	GET	`/vehicles`	Dispatch	JWT
16	GET	`/vehicles/:id`	Dispatch	JWT
17	GET	`/vehicles/:id/location`	Dispatch	JWT
18	POST	`/vehicles/:id/location`	Dispatch	None
19	PUT	`/vehicles/:id/status`	Dispatch	None
20	GET	`/vehicles/:id/location-history`	Dispatch	JWT
21	POST	`/stations`	Dispatch	JWT
22	GET	`/stations`	Dispatch	JWT
23	GET	`/stations/:id`	Dispatch	JWT
24	PUT	`/stations/:id`	Dispatch	JWT
25	PUT	`/stations/:id/capacity`	Dispatch	JWT
26	GET	`/health`	Dispatch	Public

#	Method	Path	Service	Auth
27	GET	`/analytics/dashboard`	Analytics	JWT
28	GET	`/analytics/response-times`	Analytics	JWT
29	GET	`/analytics/incidents-by-region`	Analytics	JWT
30	GET	`/analytics/incidents-by-type`	Analytics	JWT
31	GET	`/analytics/resource-utilization`	Analytics	JWT
32	GET	`/analytics/most-deployed`	Analytics	JWT
33	GET	`/analytics/events`	Analytics	JWT
34	GET	`/health`	Analytics	Public

12. Startup & Developer Guide

12.1 Prerequisites

- Node.js ≥ 18 (v25 on Windows requires the mongodb+srv fix described in Section 3.1)
- MongoDB Atlas account with IP whitelist configured (or local mongod)
- CloudAMQP account, or local RabbitMQ: docker run -d -p 5672:5672 rabbitmq:management

12.2 First-Time Setup

```
# 1. Enter project root
cd course-project

# 2. Install all service dependencies
npm run install:all

# 3. Verify .env exists at root with correct MONGODB_URI and RABBITMQ_URL

# 4. Start all 4 services (hot-reload via nodemon)
npm run dev
```

12.3 NPM Scripts Reference

Script	Description
npm run dev	Start all 4 services concurrently with colour-coded output (AUTH/INCIDENT/DISPATCH/ANALYTICS)
npm start	Production start of all 4 services
npm run install:all	Install node_modules in each service directory
npm run auth:dev	Start only Auth Service with nodemon
npm run incident:dev	Start only Incident Service with nodemon
npm run dispatch:dev	Start only Dispatch Service with nodemon
npm run analytics:dev	Start only Analytics Service with nodemon

12.4 Expected Startup Log

```
[AUTH]       Auth Service running on port 3001
[AUTH]       Swagger docs: http://localhost:3001/api-docs
[AUTH]       [RabbitMQ] Connected to emergency_platform exchange

[INCIDENT]   Incident Service running on port 3002
[INCIDENT]   Swagger docs: http://localhost:3002/api-docs

[DISPATCH]   Dispatch Service running on port 3003
[DISPATCH]   Swagger docs: http://localhost:3003/api-docs
[DISPATCH]   Socket.IO ready for real-time connections

[ANALYTICS] Analytics Service running on port 3004
```

```
[ANALYTICS] Swagger docs: http://localhost:3004/api-docs
[ANALYTICS] [Analytics] RabbitMQ consumers started
```

12.5 Quick Test Flow

- Register a user: POST /auth/register
- Login: POST /auth/login → copy accessToken
- Register a vehicle: POST /vehicles/register (use token)
- Create an incident: POST /incidents (use token) — auto-dispatch fires immediately
- View analytics: GET /analytics/dashboard — populated from RabbitMQ events
- Connect a WebSocket client to ws://localhost:3003 and watch for vehicle:location and vehicle:dispatched events

Swagger UI

Each service exposes interactive API documentation at /api-docs. You can authenticate with a Bearer token directly in the Swagger UI by clicking the Authorize button and pasting your JWT access token.